2. JS执行机制

上文回顾

在这节课开始之前，让我们先来对上一节课的内容进行一次回顾。

在上一节的课程中，我们着重学习了浏览器的导航流程，在导航中，我们对浏览器的多进程架构进行了讲解，知悉了**多进程**的优势与多线程并行处理的优势，体现在导航流程中，我们可以看到浏览器进程、网络进程与渲染进程的相互协作。在浏览器进程处理完用户输入URL后，网络进程接手任务，对网络资源发起请求。在网络进程这一块，我们插入了**TCP/IP**的知识点，讲解了dns、content-type等额外的内容。在网络进程接受到请求头后，网络进程通知浏览器进程准备渲染，随后浏览器进程对旧内容页面进行移除，渲染进程加入队伍。随着资源的加载完毕，我们接着重点讲述了渲染进程的每一个子阶段，其中包括：**构建DOM树、样式计算、布局树、布局计算、分层、图层绘制、栅格化、以及最后的合成与显示。**

接下来，让我们先补充一下上一节中剩余部分：关于重排、重绘与合成的问题。

接下来我们开始这节分享的主要内容，这节分享主要讲解内容为JS的执行机制。

变量提升

今天在该模块的第一节，我们主要讲解执行上下文相关的内容。那为什么先讲执行上下文呢？它这么重要吗？可以这么说，只有理解了 JavaScrip 的执行上下文，你才能更好地理解 JavaScript 语言本身，比如变量提升、作用域和闭包等。本节会简单的讲解JS是怎么运行的，在开始之前，先来提出一个问题：JS是按顺序执行的吗？

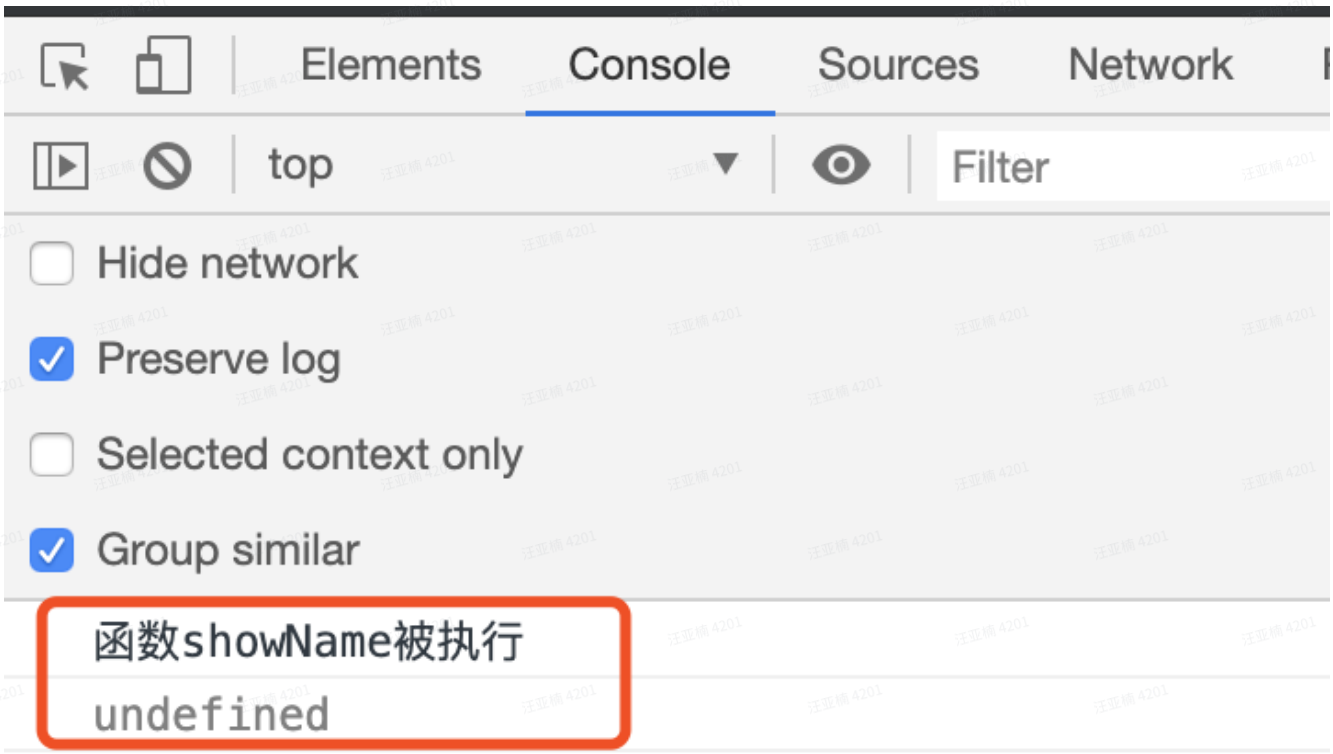
JavaScript

```
1  showName()
2  console.log(myname)
3  var myname = 'IDD'
4  function showName() {
5      console.log('函数showName被执行');
6  }
```

我们知道：JavaScript 是按顺序执行的。若按照这个逻辑来理解的话，那么：

1. 当执行到第 1 行的时候，由于函数 showName 还没有定义，所以执行应该会报错；
2. 同样执行第 2 行的时候，由于变量 myname 也未定义，所以同样也会报错。

但从打印的结果来看，却不是这样：



在上面的执行结果出来之前，你应该已经知道了函数或者变量可以在定义之前使用，我们称之为变量提升。同时，我们还知道：

- 在执行过程中，若使用了未声明的变量，那么 JavaScript 执行会报错。
- 在一个变量定义之前使用它，不会出错，但是该变量的值会为 undefined，而不是定义时的值。
- 在一个函数定义之前使用它，不会出错，且函数能正确执行。

那么这里抛出两个问题：

1. 变量和函数为什么能在其定义之前使用？这似乎表明 JavaScript 代码并不是一行一行执行的。
2. 同样的方式，变量和函数的处理结果为什么不一样？比如上面的执行结果，提前使用的 showName 函数能打印出来完整结果，但是提前使用的 myname 变量值却是 undefined，而不

是定义时使用的“IDD”这个值。

关于第一个问题，同学们应该都知道：**所谓的变量提升，是指在 JavaScript 代码执行过程中，JavaScript 引擎把变量的声明部分和函数的声明部分提升到代码开头的“行为”。变量被提升后，会给变量设置默认值，这个默认值就是我们熟悉的 undefined。**

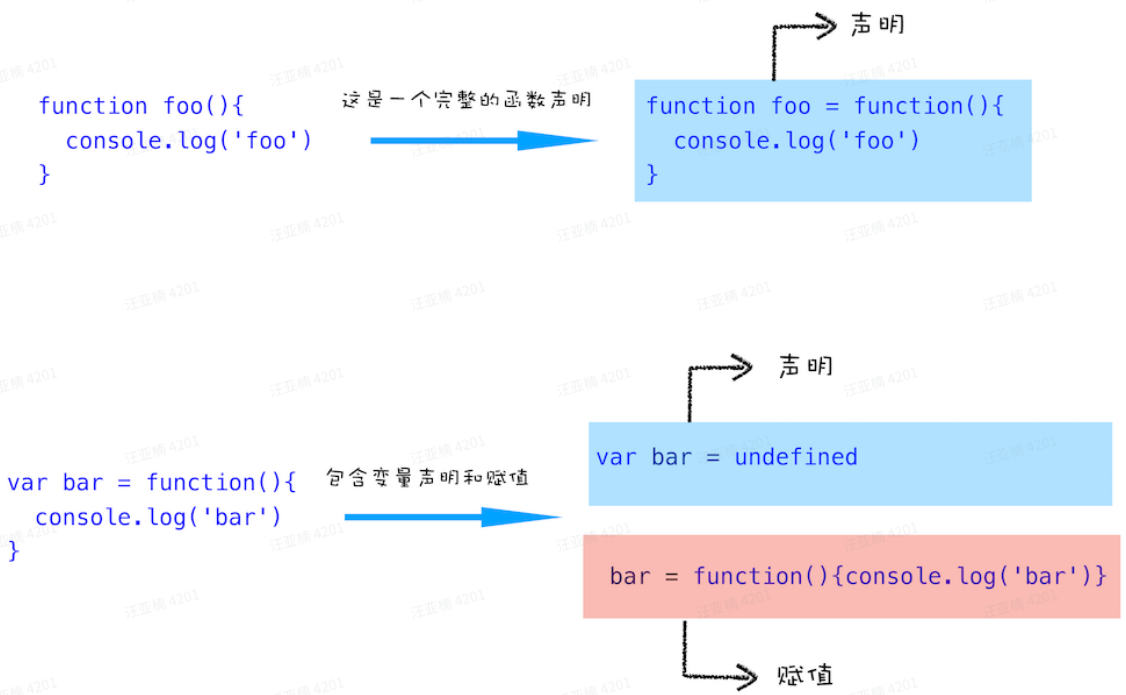
而第二个问题，我们需要搞清楚什么是函数声明和赋值。

JavaScript

```javascript
function foo(){
  console.log('foo')
}

var bar = function(){
  console.log('bar')
}
```

第一个函数 foo 是一个完整的函数声明，也就是说没有涉及到赋值操作；第二个函数是先声明变量 bar，再把function(){console.log('bar')}赋值给 bar。为了直观理解，你可以参考下图：



而之前showName的函数属于第一种：函数声明，所以整个函数声明被提升到代码开头，符合变量提升的含义。

JS的执行流程

从概念的字面意义上来看，“变量提升”意味着变量和函数的声明会在物理层面移动到代码的最前面，正如我们之前说的那样。但，这并不准确。实际上变量和函数声明在代码里的位置是不会改变

的，而是在编译阶段被 JavaScript 引擎放入内存中。对，你没听错，一段 JavaScript 代码在执行之前需要被 JavaScript 引擎编译，编译完成之后，才会进入执行阶段。大致流程你可以参考下图

1. 编译阶段

那么编译阶段和变量提升存在什么关系呢？为了搞清楚这个问题，我们还是回过头来看上面那段模拟变量提升的代码，为了方便介绍，可以把这段代码分成两部分。

第一部分：变量提升部分的代码。

JavaScript

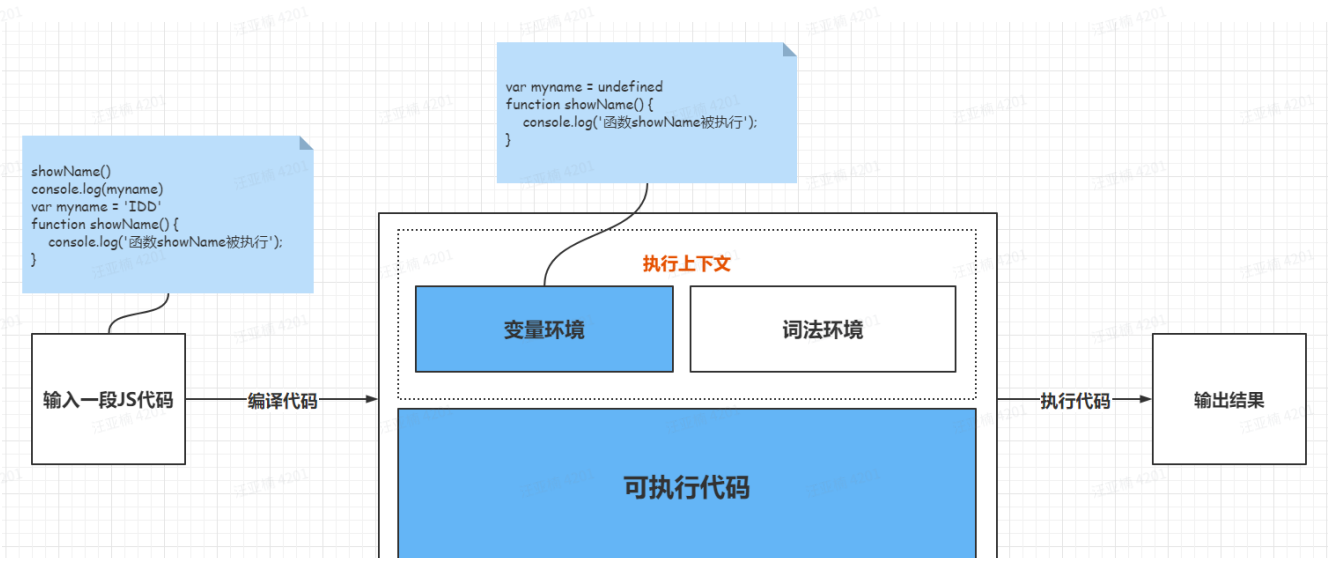
```javascript
var myname = undefined
function showName() {
    console.log('函数showName被执行');
}
```

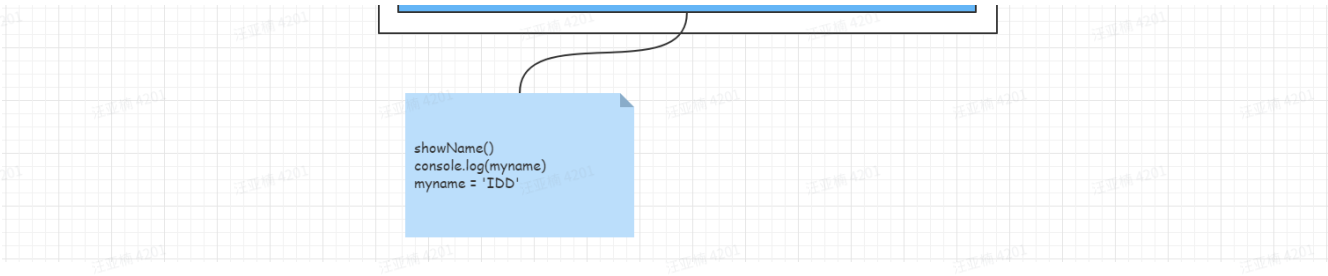
第二部分：执行部分的代码。

CoffeeScript

```coffeescript
showName()
console.log(myname)
myname = 'IDD'
```

下面我们就可以把 JavaScript 的执行流程细化，如下图所示：





从上图可以看出，输入一段代码，经过编译后，会生成两部分内容：**执行上下文（Execution context）和可执行代码。**

执行上下文是 JavaScript 执行一段代码时的运行环境，比如调用一个函数，就会进入这个函数的执行上下文，确定该函数在执行期间用到的诸如 this、变量、对象以及函数等。现在你需要知道，在执行上下文中存在一个变量环境的对象（Viriable Environment），该对象中保存了变量提升的内容，比如上面代码中的变量 myname 和函数 showName，都保存在该对象中。你可以简单地把变量环境对象看成是如下结构：

JavaScript

```
VariableEnvironment:
      myname -> undefined,
      showName ->function : {console.log(myname)
```

了解完变量环境对象的结构后，接下来，我们再结合下面这段代码来分析下是如何生成变量环境对象的。

JavaScript

```
showName()
console.log(myname)
var myname = 'IDD'
function showName() {
    console.log('函数showName被执行');
}
```

我们可以一行一行来分析上述代码：

1. 第 3 行，由于这行是经过 var 声明的，因此 JavaScript 引擎将在环境对象中创建一个名为 myname 的属性，并使用 undefined 对其初始化；
2. 第 4 行，JavaScript 引擎发现了一个通过 function 定义的函数，所以它将函数定义存储到堆(HEAP）中，并在环境对象中创建一个 showName 的属性，然后将该属性值指向堆中函数的位置。

这样就生成了变量环境对象。接下来 JavaScript 引擎会把声明以外的代码编译为字节码。你可以类比如下的模拟代码：

CoffeeScript

```
showName()
console.log(myname)
myname = 'IDD'
```

2. 执行阶段

JavaScript 引擎开始执行“可执行代码”，按照顺序一行一行地执行。下面我们就来一行一行分析下这个执行过程：

1. 当执行到 showName 函数时，JavaScript 引擎便开始在变量环境对象中查找该函数，由于变量环境对象中存在该函数的引用，所以 JavaScript 引擎便开始执行该函数，并输出“函数 showName 被执行”结果。
2. 接下来打印“myname”信息，JavaScript 引擎继续在变量环境对象中查找该对象，由于变量环境存在 myname 变量，并且其值为 undefined，所以这时候就输出 undefined。
3. 接下来执行第 3 行，把“IDD”赋给 myname 变量，赋值后变量环境中的 myname 属性值改变为“IDD”，变量环境如下所示：

JavaScript

```
VariableEnvironment:
    myname -> "IDD",
    showName ->function : {console.log(myname)
```

好了，以上就是一段代码的编译和执行流程。实际上，编译阶段和执行阶段都是非常复杂的，包括了词法分析、语法解析、代码优化、代码生成等。顺带说一下，上周晓俊分享的babel插件与这个也有关。

调用栈

那么接下来我们就来明确下，哪些情况下代码才算是“一段”代码，才会在执行之前就进行编译并创建执行上下文。一般说来，有这么三种情况：

- 当 JavaScript 执行全局代码的时候，会编译全局代码并创建全局执行上下文，而且在整个页面的生存周期内，**全局执行上下文只有一份**。
- 当调用一个函数的时候，函数体内的代码会被编译，并创建函数执行上下文，一般情况下，函数执行结束之后，创建的函数执行上下文会被销毁。
- 当使用 eval 函数的时候，eval 的代码也会被编译，并创建执行上下文。

在开始讲解下一个知识点之前，我们先看一个这样一个错误：

```
▶Uncaught RangeError: Maximum call stack size exceeded
    at division (<anonymous>:1:18)
    at division (<anonymous>:2:12)
    at division (<anonymous>:2:12)
    at division (<anonymous>:2:12)
    at division (<anonymous>:2:12)
    at division (<anonymous>:2:12)
    at division (<anonymous>:2:12)
    at division (<anonymous>:2:12)
    at division (<anonymous>:2:12)
    at division (<anonymous>:2:12)
>
```

可以看出，这是一个超出栈提示，那么超出栈提示跟我们这节执行上下文有什么联系呢？

这就涉及到了**调用栈**的内容。你应该知道 JavaScript 中有很多函数，经常会出现在一个函数中调用另外一个函数的情况，**调用栈就是用来管理函数调用关系的一种数据结构**。因此要讲清楚调用栈，你还要先弄明白**函数调用**和**栈结构**。

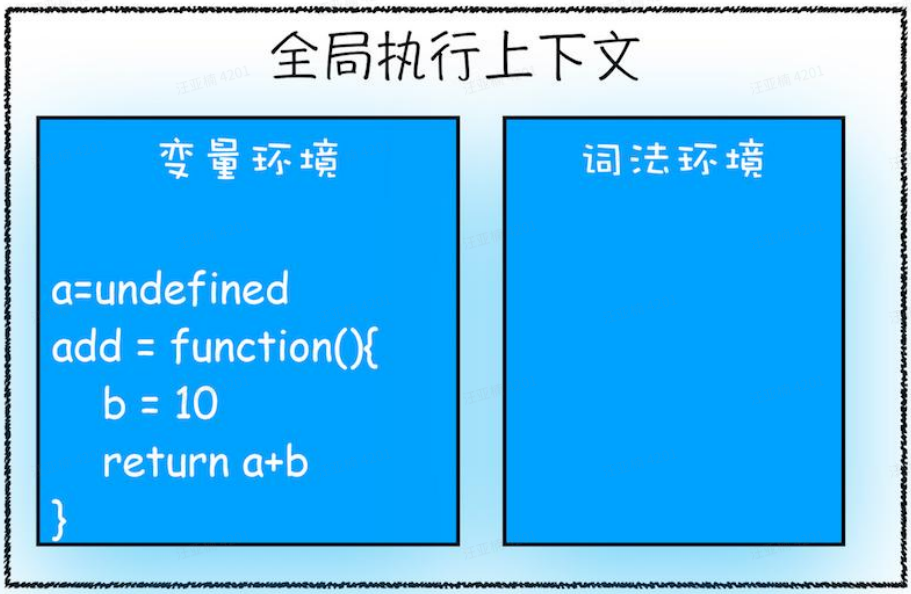
什么是函数调用

函数调用就是运行一个函数，具体使用方式是使用函数名称跟着一对小括号。下面我们看个简单的示例代码：

JavaScript

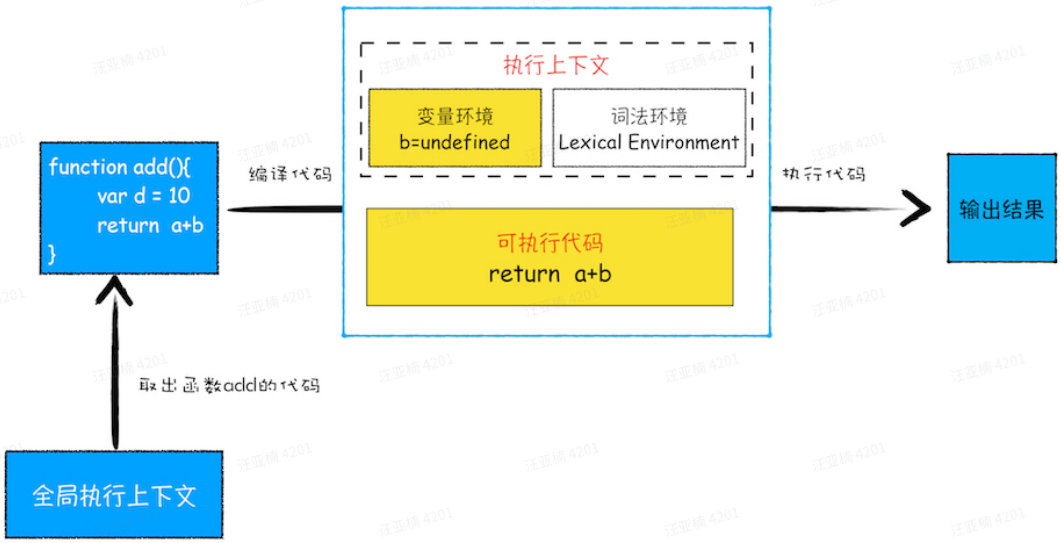
```javascript
var a = 2
function add(){
var b = 10
return  a+b
}
add()
```

在执行到函数 add() 之前，JavaScript 引擎会为上面这段代码创建全局执行上下文，包含了声明的函数和变量，你可以参考下图：



执行上下文准备好之后，便开始执行全局代码，当执行到 add 这儿时，JavaScript 判断这是一个函数调用，那么将执行以下操作：

1. 首先，从全局执行上下文中，取出 add 函数代码。
2. 其次，对 add 函数的这段代码进行编译，并创建该函数的执行上下文和可执行代码。
3. 最后，执行代码，输出结果。



就这样，当执行到 add 函数的时候，我们就有了两个执行上下文了——全局执行上下文和 add 函数的执行上下文。

也就是说在执行 JavaScript 时，可能会存在多个执行上下文，那么 JavaScript 引擎是如何管理这些执行上下文的呢？

答案是通过一种叫栈的数据结构来管理的。它是如何管理这些执行上下文呢？

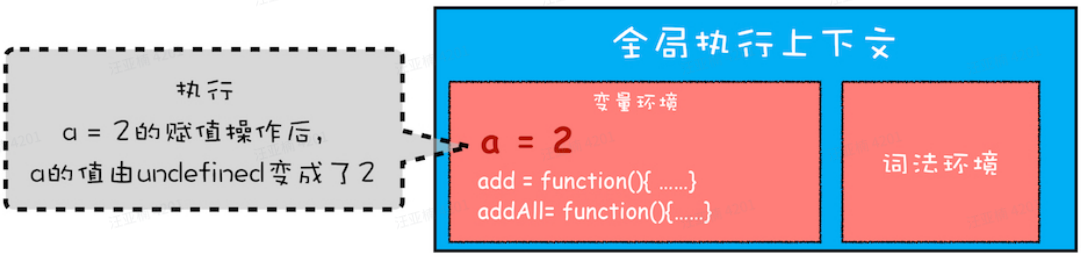
我们知道栈是一种后进先出的数据结构，JavaScript 引擎正是利用栈的这种结构来管理执行上下文的。在执行上下文创建好后，JavaScript 引擎会将执行上下文压入栈中，通常把这种用来管理执行上下文的栈称为**执行上下文栈**，又称**调用栈**。

JavaScript

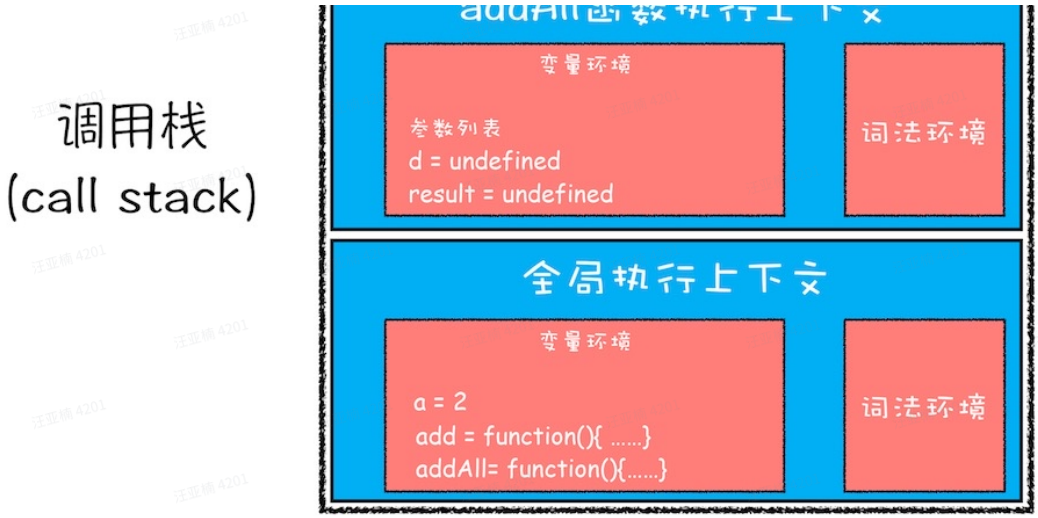
```javascript
var a = 2
function add(b,c){
  return b+c
}
function addAll(b,c){
var d = 10
result = add(b,c)
return  a+result+d
}
addAll(3,6)
```

下面我们就一步步地分析在代码的执行过程中，调用栈的状态变化情况。

全局执行上下文会首先压入到调用栈，整理好产量提升后，JavaScript 引擎便开始执行全局代码了。首先会执行 a=2 的赋值操作，执行该语句会将全局上下文变量环境中 a 的值设置为 2。设置后的全局上下文的状态如下图所示：

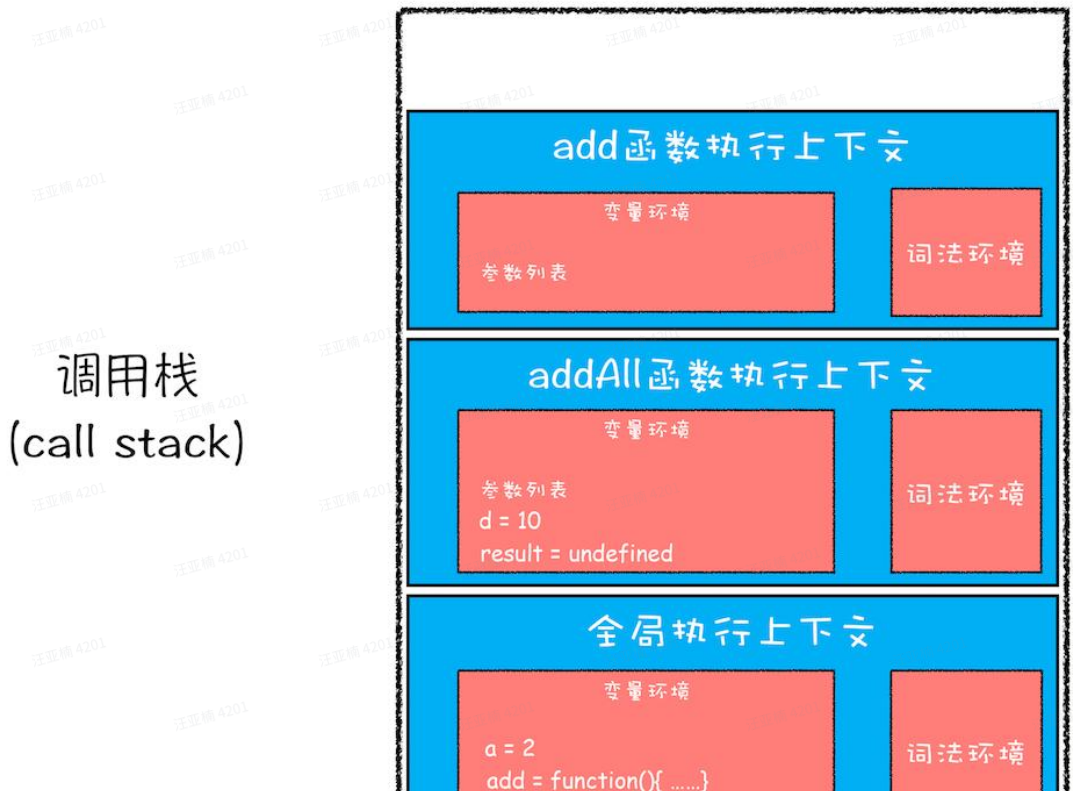


接下来，第二步是**调用 addAll 函数**。当调用该函数时，JavaScript 引擎会编译该函数，并为其创建一个执行上下文，最后还将该函数的执行上下文压入栈中，如下图所示：

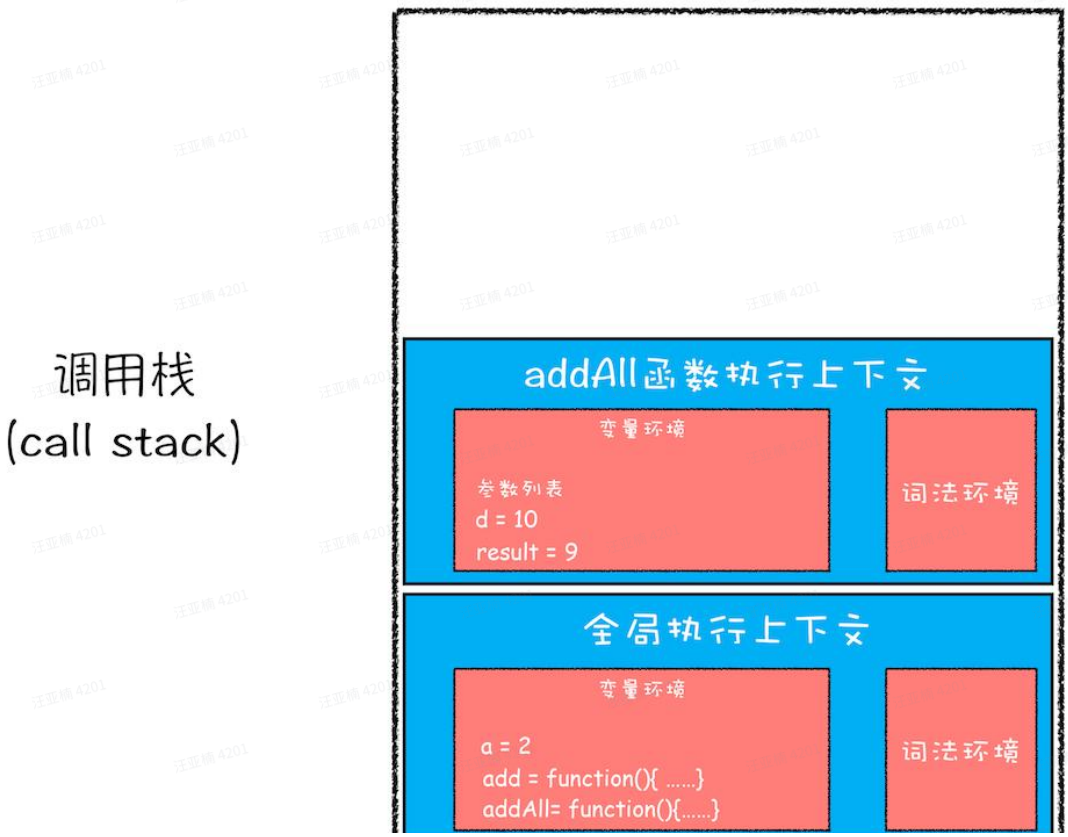


addAll 函数的执行上下文创建好之后，便进入了函数代码的执行阶段了，这里先执行的是 d=10 的赋值操作，执行语句会将 addAll 函数执行上下文中的 d 由 undefined 变成了 10。

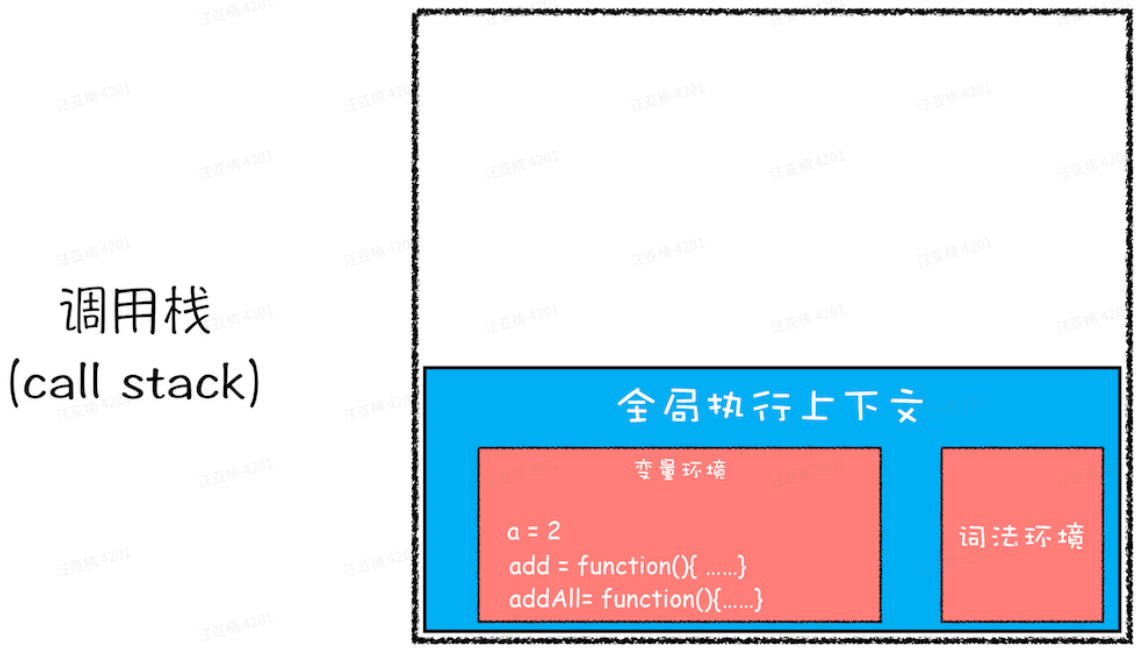
然后接着往下执行，第三步，当**执行到 add 函数**调用语句时，同样会为其创建执行上下文，并将其压入调用栈，如下图所示：



当 add 函数返回时，该函数的执行上下文就会从栈顶弹出，并将 result 的值设置为 add 函数的返回值，也就是 9。如下图所示：



紧接着 addAll 执行最后一个相加操作后并返回，addAll 的执行上下文也会从栈顶部弹出，此时调用栈中就只剩下全局上下文了。最终如下图所示：



至此，整个 JavaScript 流程执行结束了。

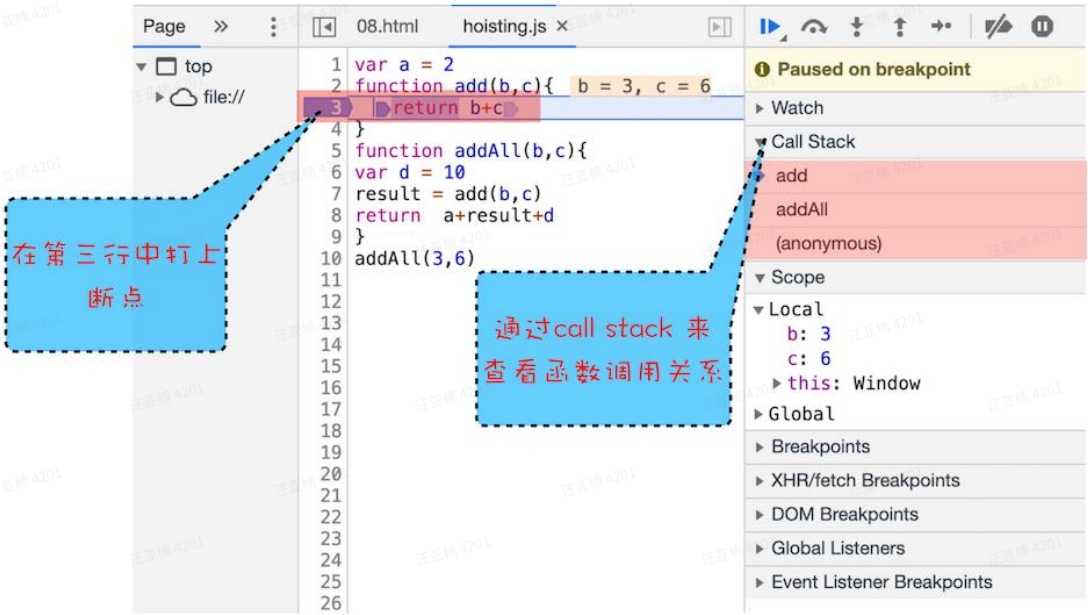
好了，现在你应该知道了**调用栈是 JavaScript 引擎追踪函数执行的一个机制**，当一次有多个函数被调用时，通过调用栈就能够追踪到哪个函数正在被执行以及各函数之间的调用关系。

在开发中，如何利用好调用栈

鉴于调用栈的重要性和实用性，那么接下来我们就一起来看看在实际工作中，应该如何查看和利用好调用栈。

如何利用浏览器查看调用栈的信息

你可以打开“开发者工具”，点击“Source”标签，选择 JavaScript 代码的页面，然后在第 3 行加上断点，并刷新页面。你可以看到执行到 add 函数时，执行流程就暂停了，这时可以通过右边“call stack”来查看当前的调用栈的情况，如下图：



从图中可以看出，右边的“call stack”下面显示出来了函数的调用关系：栈的最底部是 anonymous，也就是全局的函数入口；中间是 addAll 函数；顶部是 add 函数。这就清晰地反映了函数的调用关系，所以在**分析复杂结构代码，或者检查 Bug 时，调用栈都是非常有用的。**

栈溢出（Stack Overflow）

现在你知道了调用栈是一种用来管理执行上下文的数据结构，符合后进先出的规则。不过还有一点你要注意，**调用栈是有大小的**，当入栈的执行上下文超过一定数目，JavaScript 引擎就会报错，我们把这种错误叫做**栈溢出**。

特别是在你写递归代码的时候，就很容易出现栈溢出的情况。比如下面这段代码：

JavaScript

```javascript
function division(a,b){
    return division(a,b)
}
console.log(division(1,2))
```

当执行时，就会抛出栈溢出错误。

块级作用域

前面我们讲解了 JavaScript 中变量提升的相关内容，**正是由于 JavaScript 存在变量提升这种特性，从而导致了很多与直觉不符的代码，这也是 JavaScript 的一个重要设计缺陷。**

这一节会先“探病因”——变量提升所带来的问题；然后再来“开药方”——介绍如何通过块级作用域并配合 let 和 const 关键字来修复这种缺陷。

作用域（scope）

作用域是指在程序中定义变量的区域，该位置决定了变量的生命周期。通俗地理解，作用域就是变量与函数的可访问范围，即作用域控制着变量和函数的可见性和生命周期。

在 ES6 之前，ES 的作用域只有两种：**全局作用域和函数作用域。**

变量提升所带来的问题

1. 变量容易在不被察觉的情况下被覆盖掉
2. 本应销毁的变量没有被销毁

JavaScript

```javascript
// 问题1
var myname = "IDD"
function showName(){
  console.log(myname);
  if(0){
   var myname = "前端组"
  }
  console.log(myname);
}
showName()

// 问题2
function foo(){
  for (var i = 0; i < 7; i++) {
  }
  console.log(i);
}
foo()
```

ES6 是如何解决变量提升带来的缺陷

ES6 引入了 let 和 const 关键字，从而使 JavaScript 也能像其他语言一样拥有了块级作用域。

JavaScript

```javascript
function letTest() {
  let x = 1;
  if (true) {
    let x = 2;  // 不同的变量
    console.log(x);  // 2
  }
  console.log(x);  // 1
}
```

执行这段代码，其输出结果就和我们的预期是一致的。这是因为 let 关键字是支持块级作用域的，所以在编译阶段，JavaScript 引擎并不会把 if 块中通过 let 声明的变量存放到变量环境中，这也就意味着在 if 块通过 let 声明的关键字，并不会提升到全函数可见。所以在 if 块之内打印出来的值是 2，跳出语块之后，打印出来的值就是 1 了。这种就非常符合我们的编程习惯了：**作用域块内声明的变量不影响块外面的变量。**

JavaScript 是如何支持块级作用域的

JavaScript 是如何支持块级作用域的

现在你知道了 ES 可以通过使用 let 或者 const 关键字来实现块级作用域，不过你是否有过这样的疑问：“在同一段代码中，ES6 是如何做到既要支持变量提升的特性，又要支持块级作用域的呢？”

那么接下来，我们就要站在**执行上下文**的角度来揭开答案。

你已经知道 JavaScript 引擎是通过变量环境实现函数级作用域的，那么 ES6 又是如何在函数级作用域的基础之上，实现对块级作用域的支持呢？你可以先看下面这段代码：

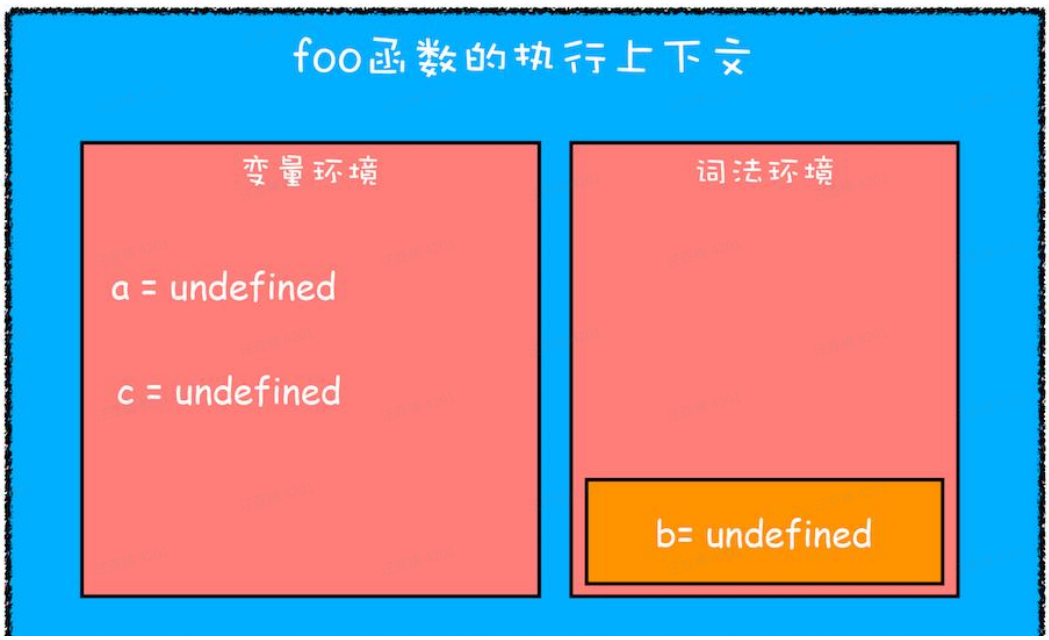
JavaScript

```javascript
function foo(){
    var a = 1
    let b = 2
    {
      let b = 3
      var c = 4
      let d = 5
      console.log(a)
      console.log(b)
    }
    console.log(b)
    console.log(c)
    console.log(d)
}
foo()
```

关于如何创建执行上下文我们在前面已经分析过了，但是现在的情况有点不一样，我们引入了 let 关键字，let 关键字会创建块级作用域，那么 let 关键字是如何影响执行上下文的呢？

接下来我们就来一步步分析上面这段代码的执行流程。

第一步是**编译并创建执行上下文**，下面是执行上下文示意图，你可以参考下：

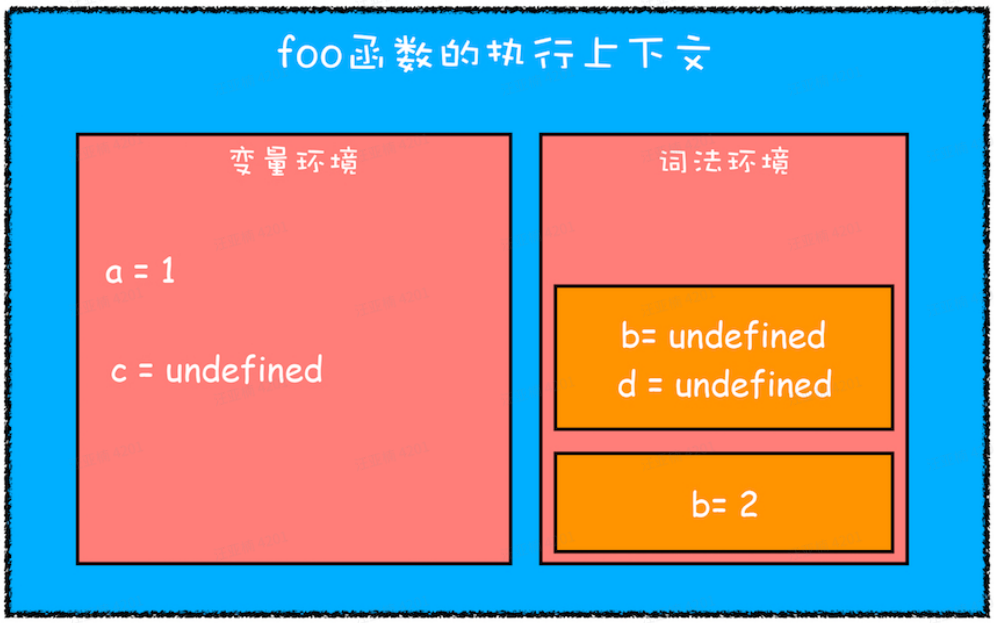


刚执行时 foo 函数的执行上下文

通过上图，我们可以得出以下结论：

- 函数内部通过 var 声明的变量，在编译阶段全都被存放到变量环境里面了。
- 通过 let 声明的变量，在编译阶段会被存放到词法环境（Lexical Environment）中。
- 在函数的作用域内的块级作用域内部（代码中5-7行），通过 let 声明的变量并没有被存放到词法环境中。

接下来，第二步继续执行代码，当执行到代码块里面时，变量环境中 a 的值已经被设置成了 1，词法环境中 b 的值已经被设置成了 2，这时候函数的执行上下文就如下图所示：

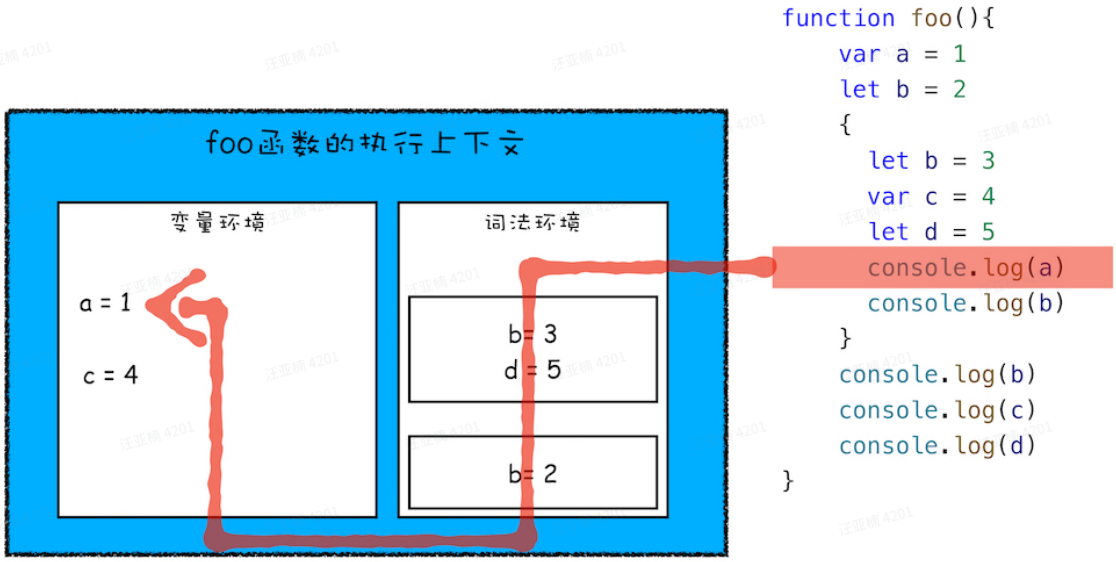


从图中可以看出，当进入函数的作用域块时，作用域块中通过 let 声明的变量，会被存放在词法环境的一个单独的区域中，这个区域中的变量并不影响作用域块外面的变量，比如在作用域外面声明了变量 b，在该作用域块内部也声明了变量 b，当执行到作用域内部时，它们都是独立的存在。

其实，在词法环境内部，维护了一个小型栈结构，栈底是函数最外层的变量，进入一个作用域块后，就会把该作用域块内部的变量压到栈顶；当作用域执行完成之后，该作用域的信息就会从栈顶弹出，这就是词法环境的结构。需要注意下，我这里所讲的变量是指通过 let 或者 const 声明的变量。

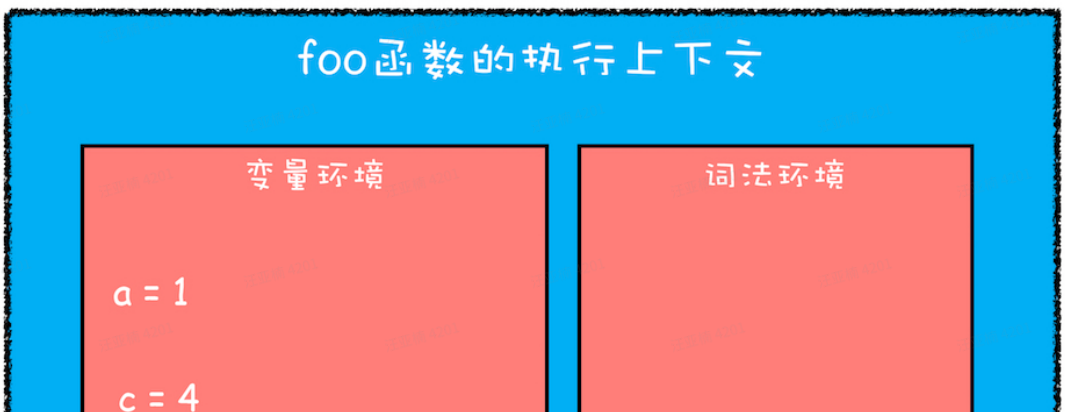
再接下来，当执行到作用域块中的console.log(a)这行代码时，就需要在词法环境和变量环境中查找变量 a 的值了，具体查找方式是：沿着词法环境的栈顶向下查询，如果在词法环境中的某个块中查找到了，就直接返回给 JavaScript 引擎，如果没有查找到，那么继续在变量环境中查找。

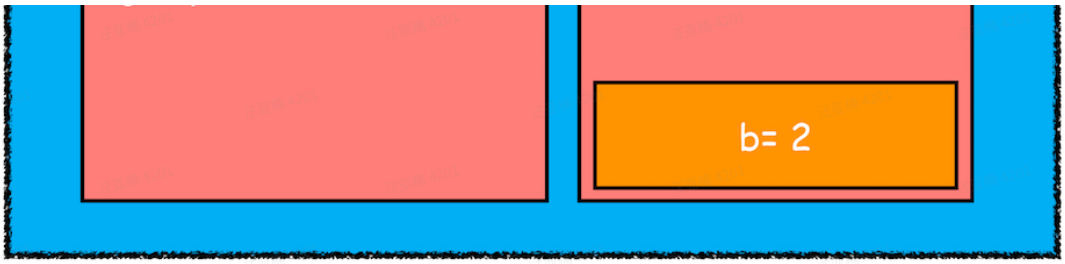
这样一个变量查找过程就完成了，你可以参考下图：



从上图你可以清晰地看出变量查找流程，不过要完整理解查找变量或者查找函数的流程，就涉及到作用域链了，关于这个内容会在接下来的小节中讲解。

当作用域块执行结束之后，其内部定义的变量就会从词法环境的栈顶弹出，最终执行上下文如下图所示：





通过上面的分析，想必你已经理解了词法环境的结构和工作机制，块级作用域就是通过词法环境的栈结构来实现的，而变量提升是通过变量环境来实现，通过这两者的结合，JavaScript 引擎也就同时支持了变量提升和块级作用域了。

作用域链与闭包

在上一节中我们提到了如何通过词法环境和变量环境来查找变量，这其中就涉及到作用域链的概念。理解作用域链是理解闭包的基础，而闭包在 JavaScript 中几乎无处不在，同时作用域和作用域链还是所有编程语言的基础。

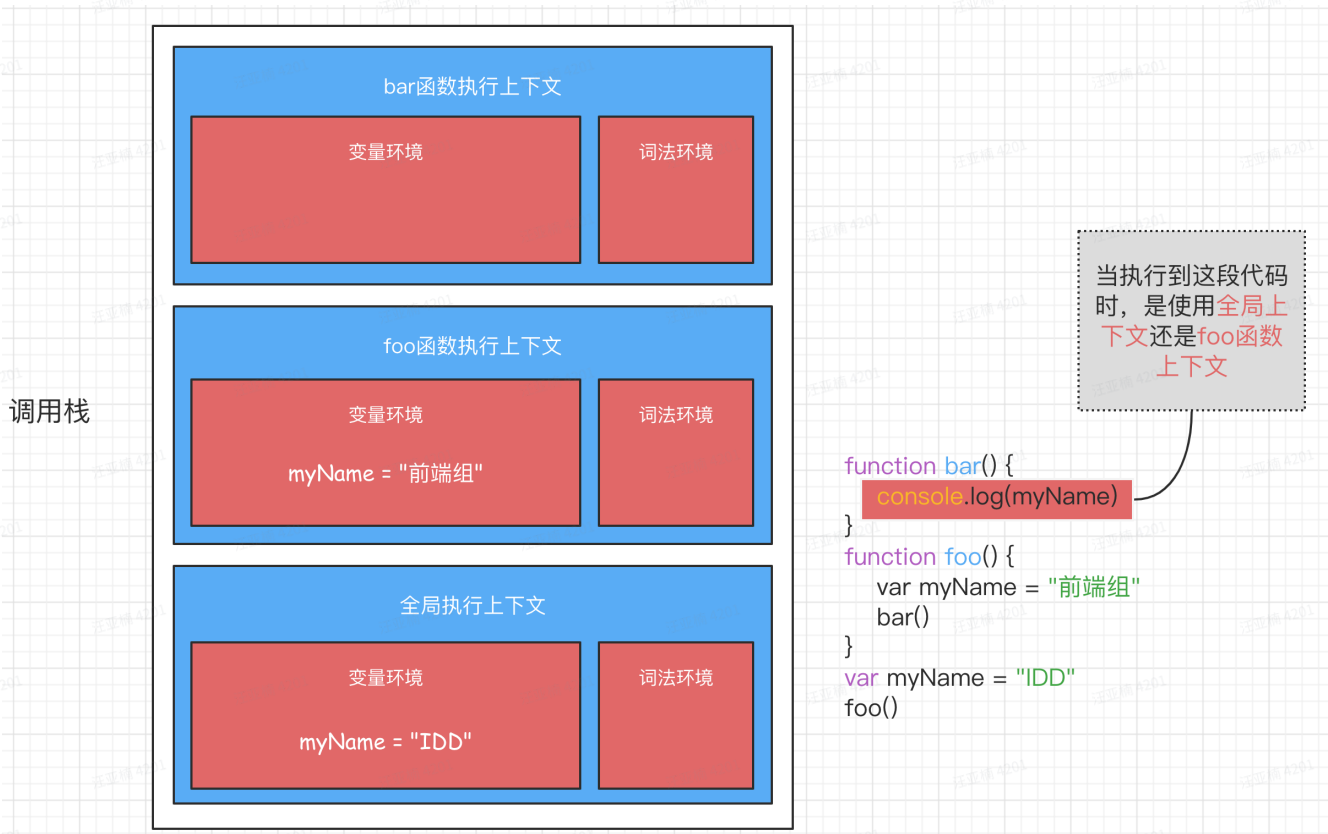
那我们就来聊聊**什么是作用域链**，并通过作用域链再来讲讲**什么是闭包**。首先我们来看下面这段代码

JavaScript

```javascript
function bar() {
    console.log(myName)
}
function foo() {
    var myName = "前端组"
    bar()
}
var myName = "IDD"
foo()
```

你觉得这段代码中的 bar 函数和 foo 函数打印出来的内容是什么？这就要分析下这两段代码的执行流程。

通过前面几篇文章的学习，想必你已经知道了如何通过执行上下文来分析代码的执行流程了。那么当这段代码执行到 bar 函数内部时，其调用栈的状态图如下所示：



从图中可以看出，全局执行上下文和 foo 函数的执行上下文中都包含变量 myName，那 bar 函数里面 myName 的值到底该选择哪个呢？

也许你的第一反应是按照调用栈的顺序来查找变量，查找方式如下：

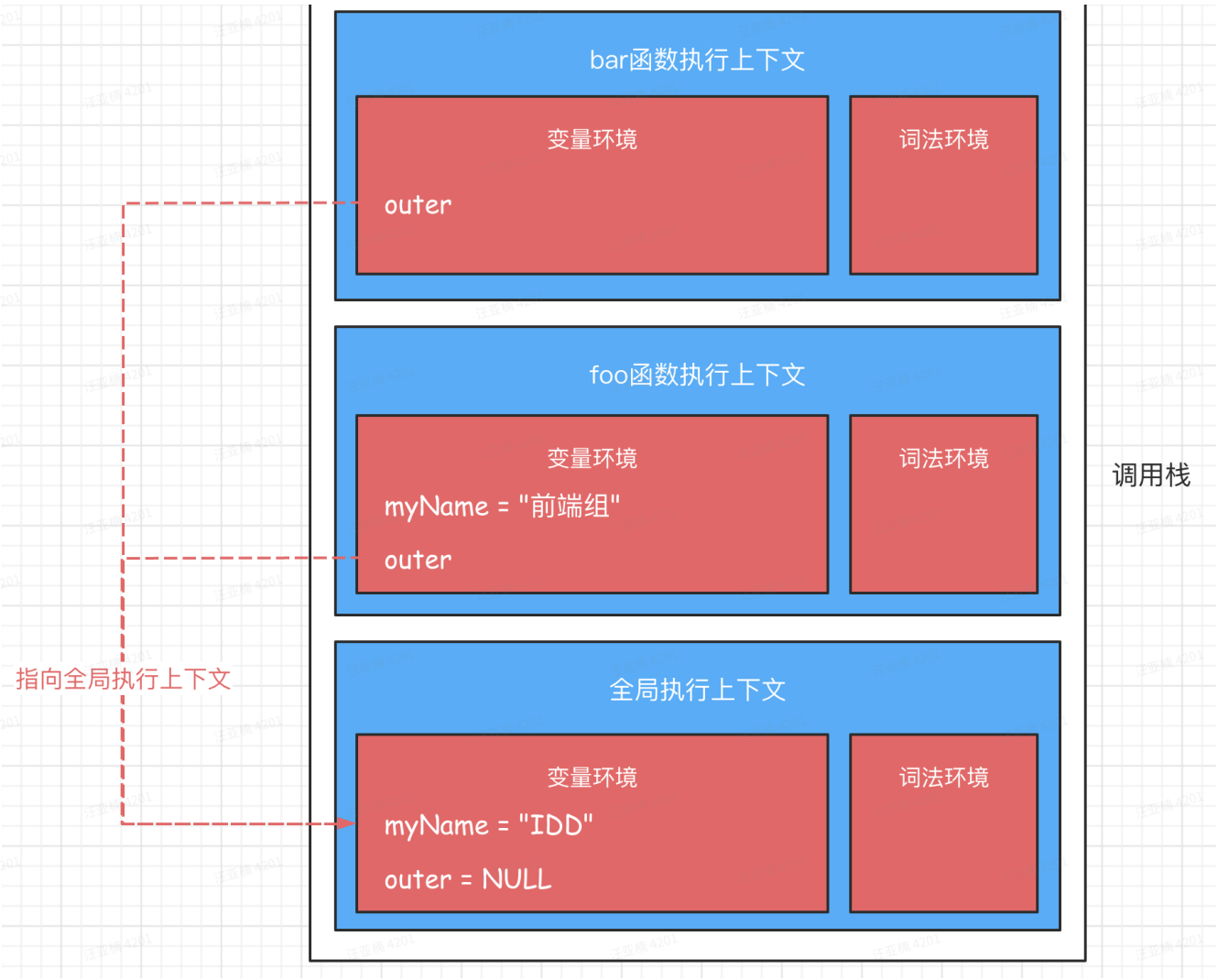
1. 先查找栈顶是否存在 myName 变量，但是这里没有，所以接着往下查找 foo 函数中的变量。
2. 在 foo 函数中查找到了 myName 变量，这时候就使用 foo 函数中的 myName。

如果按照这种方式来查找变量，那么最终执行 bar 函数打印出来的结果就应该是“前端组”。但实际情况并非如此，如果你试着执行上述代码，你会发现打印出来的结果是“IDD”。为什么会是这种情况呢？要解释清楚这个问题，那么你就需要先搞清楚作用域链了。

作用域链

其实在每个执行上下文的变量环境中，都包含了一个外部引用，用来指向外部的执行上下文，我们把这个外部引用称为 **outer**。

当一段代码使用了一个变量时，JavaScript 引擎首先会在“当前的执行上下文”中查找该变量，比如上面那段代码在查找 myName 变量时，如果在当前的变量环境中没有查找到，那么 JavaScript 引擎会继续在 outer 所指向的执行上下文中查找。为了直观理解，你可以看下面这张图：



从图中可以看出，bar 函数和 foo 函数的 outer 都是指向全局上下文的，这也就意味着如果在 bar 函数或者 foo 函数中使用了外部变量，那么 JavaScript 引擎会去全局执行上下文中查找。我们把这个查找的链条就称为**作用域链**。

现在你知道变量是通过作用域链来查找的了，不过还有一个疑问没有解开，foo 函数调用的 bar 函数，那为什么 bar 函数的外部引用是全局执行上下文，而不是 foo 函数的执行上下文？

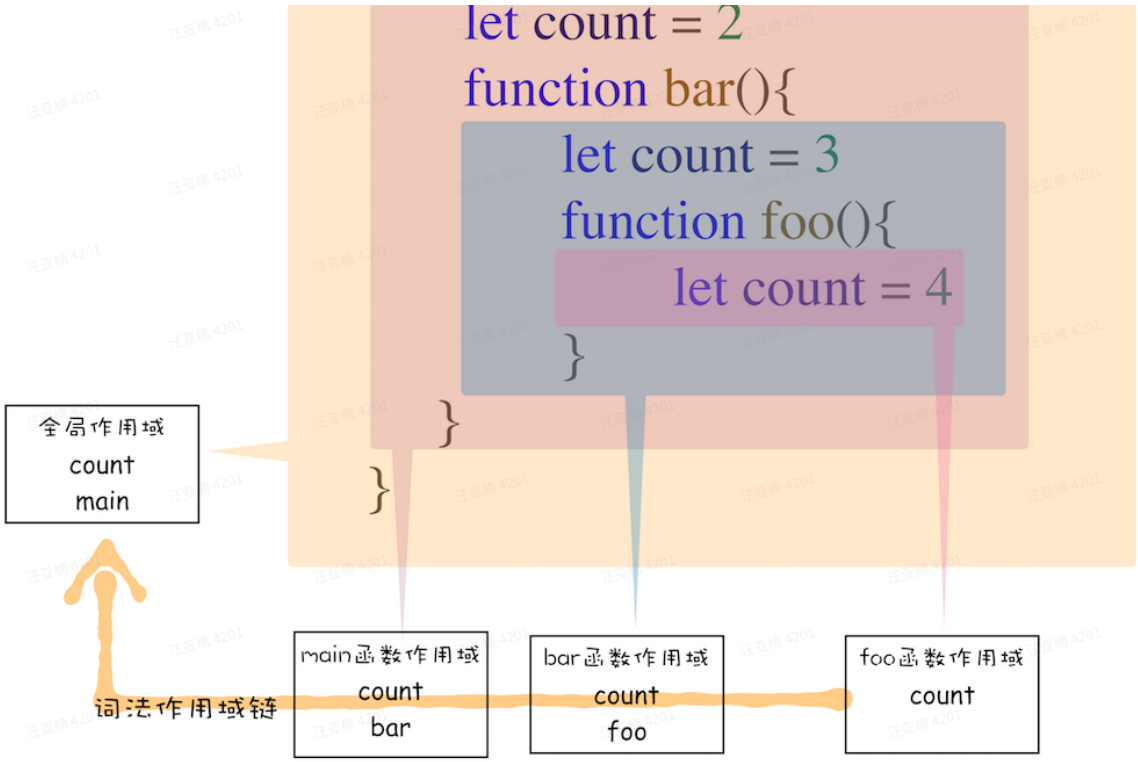
要回答这个问题，你还需要知道什么是**词法作用域**。这是因为在 JavaScript 执行过程中，其作用域链是由词法作用域决定的。

词法作用域

词法作用域就是指作用域是由代码中函数声明的位置来决定的，所以词法作用域是静态的作用域，通过它就能够预测代码在执行过程中如何查找标识符。

这么讲可能不太好理解，你可以看下面这张图：

```
let count = 1
function main(){
```



从图中可以看出，词法作用域就是根据代码的位置来决定的，其中 main 函数包含了 bar 函数，bar 函数中包含了 foo 函数，因为 JavaScript 作用域链是由词法作用域决定的，所以整个词法作用域链的顺序是：foo 函数作用域—>bar 函数作用域—>main 函数作用域—> 全局作用域。

了解了词法作用域以及 JavaScript 中的作用域链，我们再回过头来看看上面的那个问题：在开头那段代码中，foo 函数调用了 bar 函数，那为什么 bar 函数的外部引用是全局执行上下文，而不是 foo 函数的执行上下文?

这是因为根据词法作用域，foo 和 bar 的上级作用域都是全局作用域，所以如果 foo 或者 bar 函数使用了一个它们没有定义的变量，那么它们会到全局作用域去查找。也就是说，**词法作用域是代码编译阶段就决定好的，和函数是怎么调用的没有关系。**

实践

接下来让我们结合块级作用域和作用域链实践一下：

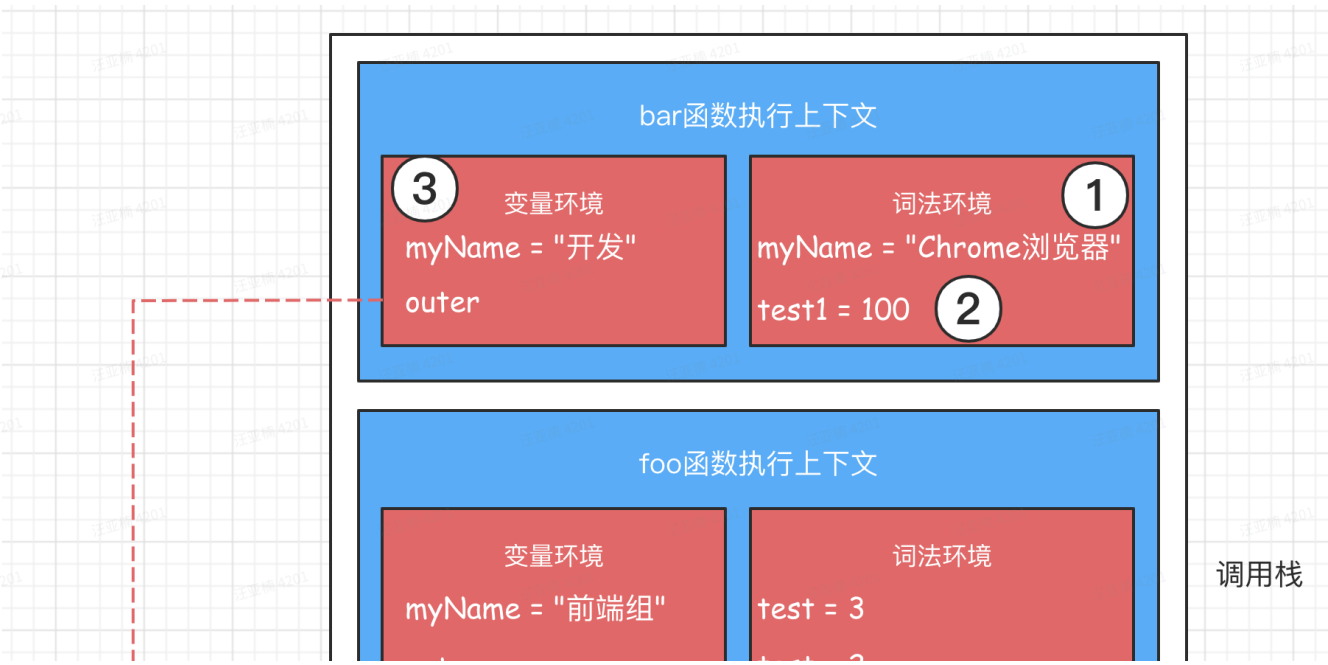
JavaScript

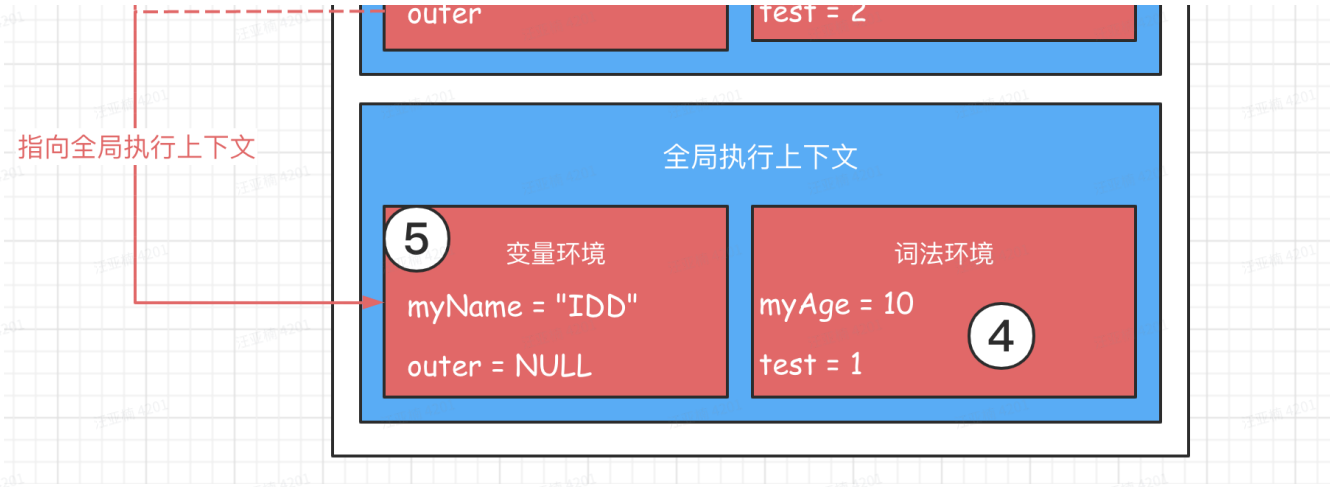
```javascript
function bar() {
    var myName = "开发"
    let test1 = 100
    if (1) {
        let myName = "Chrome浏览器"
        console.log(test)
    }
}
function foo() {
    var myName = "前端组"
    let test = 2
    {
        let test = 3
        bar()
    }
}
var myName = "IDD"
let myAge = 10
let test = 1
foo()
```

你可以自己先分析下这段代码的执行流程，看看能否分析出来执行结果。

要想得出其执行结果，那接下来我们就得站在作用域链和词法环境的角度来分析下其执行过程。

我们已经介绍过了，ES6 是支持块级作用域的，当执行到代码块时，如果代码块中有 let 或者 const 声明的变量，那么变量就会存放到该函数的词法环境中。对于上面这段代码，当执行到 bar 函数内部的 if 语句块时，其调用栈的情况如下图所示：





现在是执行到 bar 函数的 if 语块之内，需要打印出来变量 test，那么就需要查找到 test 变量的值，其查找过程我已经在上图中使用序号 1、2、3、4、5 标记出来了。

下面我就来解释下这个过程。首先是在 bar 函数的执行上下文中查找，但因为 bar 函数的执行上下文中没有定义 test 变量，所以根据词法作用域的规则，下一步就在 bar 函数的外部作用域中查找，也就是全局作用域。

在全局作用域中的词法环境中，我们找到了test = 1

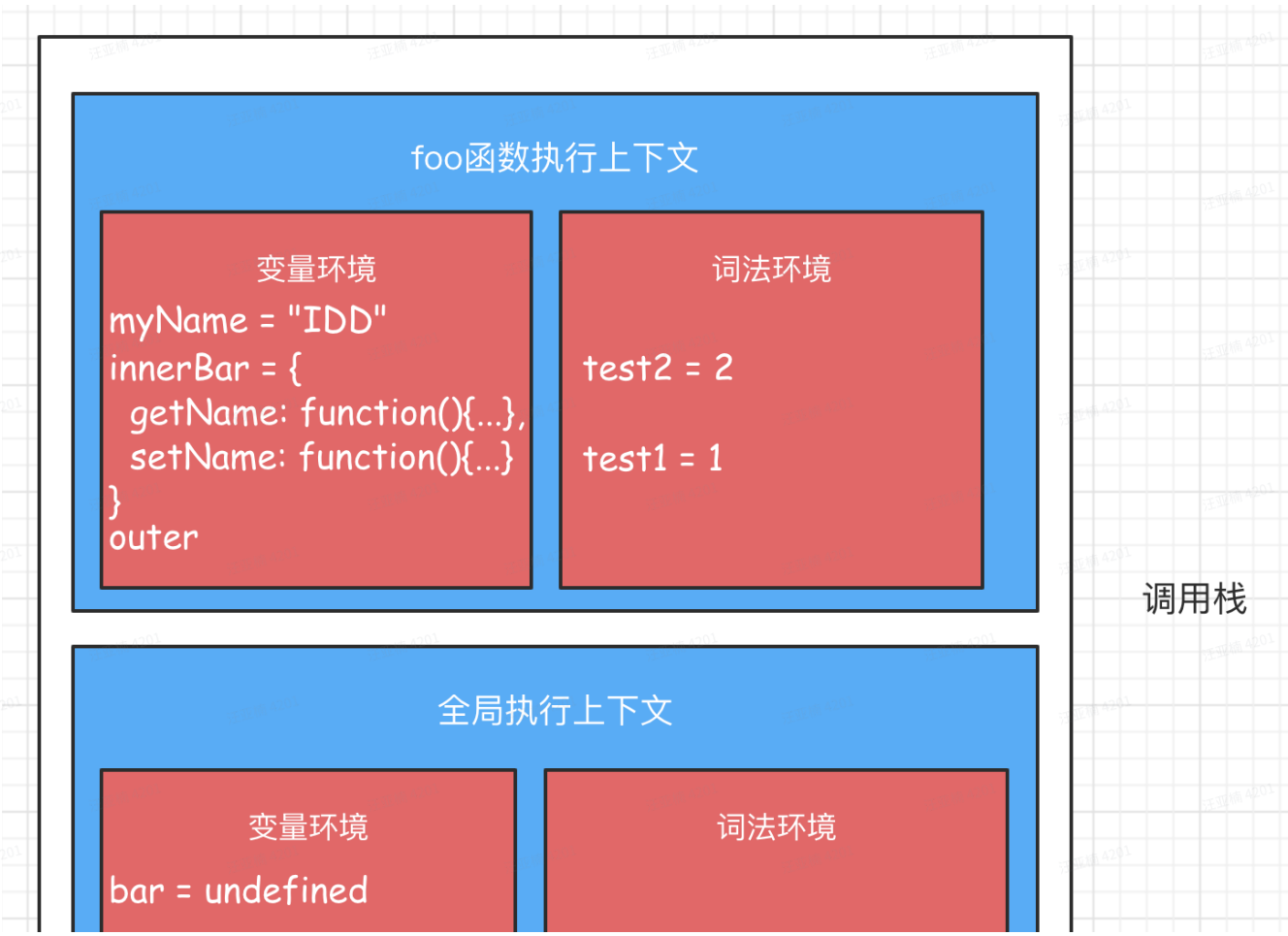
闭包

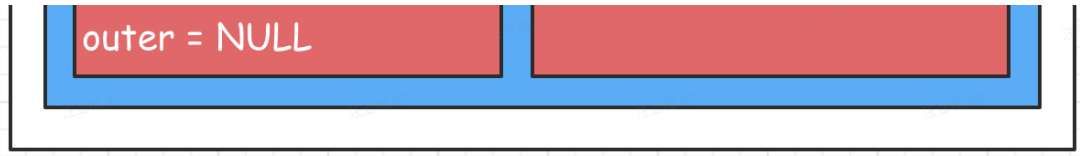
理解了变量环境、词法环境和作用域链等概念，那接下来你再理解什么是 JavaScript 中的闭包就容易多了。这里你可以结合下面这段代码来理解什么是闭包：

JavaScript

```javascript
function foo() {
    var myName = "IDD"
    let test1 = 1
    const test2 = 2
    var innerBar = {
        getName:function(){
            console.log(test1)
            return myName
        },
        setName:function(newName){
            myName = newName
        }
    }
    return innerBar
}
var bar = foo()
bar.setName("前端组")
bar.getName()
console.log(bar.getName())
```

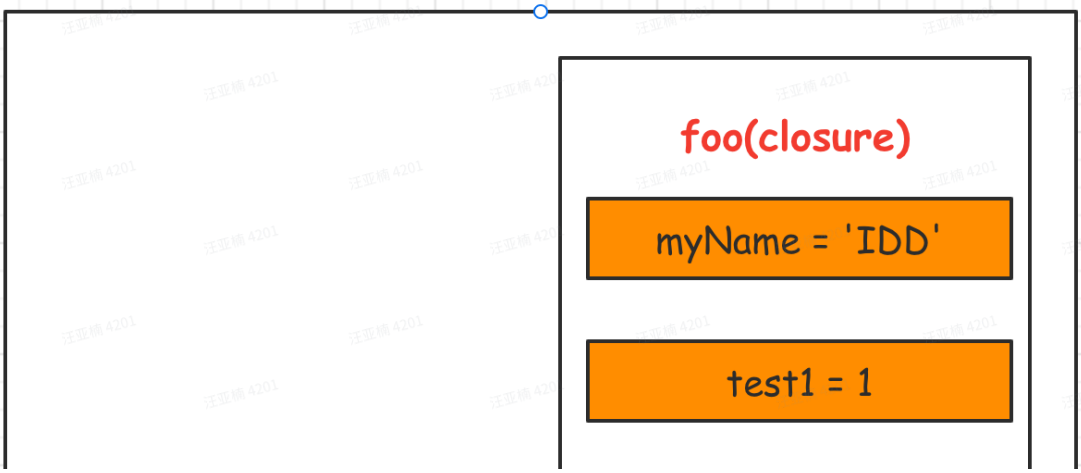
首先我们看看当执行到 foo 函数内部的return innerBar这行代码时调用栈的情况，你可以参考下图：

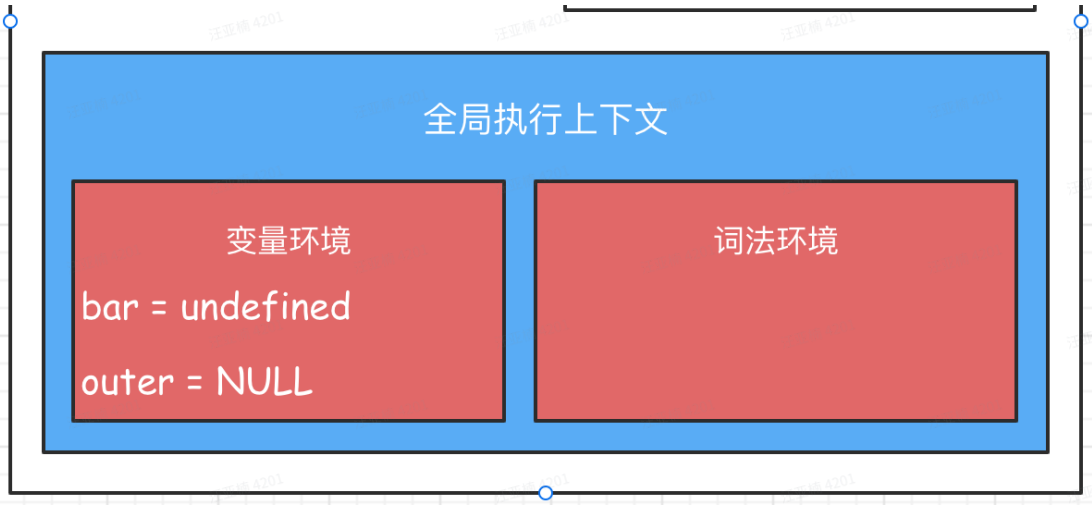




从上面的代码可以看出，innerBar 是一个对象，包含了 getName 和 setName 的两个方法（通常我们把对象内部的函数称为方法）。你可以看到，这两个方法都是在 foo 函数内部定义的，并且这两个方法内部都使用了 myName 和 test1 两个变量。

根据词法作用域的规则，内部函数 getName 和 setName 总是可以访问它们的外部函数 foo 中的变量，所以当 innerBar 对象返回给全局变量 bar 时，虽然 foo 函数已经执行结束，但是 getName 和 setName 函数依然可以使用 foo 函数中的变量 myName 和 test1。所以当 foo 函数执行完成之后，其整个调用栈的状态如下图所示：





从上图可以看出，foo 函数执行完成之后，其执行上下文从栈顶弹出了，但是由于返回的 setName 和 getName 方法中使用了 foo 函数内部的变量 myName 和 test1，所以这两个变量依然保存在内存中。这像极了 setName 和 getName 方法背的一个专属背包，无论在哪里调用了 setName 和 getName 方法，它们都会背着这个 foo 函数的专属背包。

之所以是**专属**背包，是因为除了 setName 和 getName 函数之外，其他任何地方都是无法访问该背包的，我们就可以把这个背包称为 foo 函数的**闭包**。

好了，现在我们终于可以给闭包一个正式的定义了。**在 JavaScript 中，根据词法作用域的规则，内部函数总是可以访问其外部函数中声明的变量，当通过调用一个外部函数返回一个内部函数后，即使该外部函数已经执行结束了，但是内部函数引用外部函数的变量依然保存在内存中，我们就把这些变量的集合称为闭包。比如外部函数是 foo，那么这些变量的集合就称为 foo 函数的闭包。**

那这些闭包是如何使用的呢？当执行到 bar.setName 方法中的myName = "前端组"这句代码时，JavaScript 引擎会沿着“当前执行上下文–>foo 函数闭包–> 全局执行上下文”的顺序来查找 myName 变量，你可以参考下面的调用栈状态图：

从图中可以看出，setName 的执行上下文中没有 myName 变量，foo 函数的闭包中包含了变量 myName，所以调用 setName 时，会修改 foo 闭包中的 myName 变量的值。

同样的流程，当调用 bar.getName 的时候，所访问的变量 myName 也是位于 foo 函数闭包中的。

你也可以通过“开发者工具”来看看闭包的情况，打开 Chrome 的“开发者工具”，在 bar 函数任意地方打上断点，然后刷新页面，可以看到如下内容：

```
1  function foo() {
2      var myName = "IDD"
3      let test1 = 1
4      const test2 = 2
5      var innerBar = {
6          getName:function(){
7              debugger;
8              console.log(test1)
9              return myName
10         },
11         setName:function(newName){
12             myName = newName
13         }
14     }
```

调试程序已暂停
线程
主要 已暂停
5a3e8c10-51f7-44ad-aaf0-b01a...
6a4e0e28-8b25-48db-a430-c4c...
6f3162f2-5393-44fd-93c2-ef7d7...
监视
断点

```
15    return innerBar
16    }
17    var bar = foo()
18    bar.setName("前端组")
19    bar.getName()
20    console.log(bar.getName())
```

从图中可以看出来，当调用 bar.getName 的时候，右边 Scope 项就体现出了作用域链的情况：Local 就是当前的 getName 函数的作用域，Closure(foo) 是指 foo 函数的闭包，最下面的 Global 就是指全局作用域，从“Local–>Closure(foo)–>Global”就是一个完整的作用域链。

所以说，你以后也可以通过 Scope 来查看实际代码作用域链的情况，这样调试代码也会比较方便。